Check the Tree: Finding Wrong Results in PostgreSQL with Typed Derivation Trees and Commit-Directed Grammar Extension

ANONYMOUS AUTHOR(S)

A database system that returns a wrong result raises no error and leaves no trace in its logs. Fuzzers for PostgreSQL mostly find crashes, and the oracles that detect wrong results work on small query languages of their own. In PostgreSQL 17, 18 and master, for example, the planner answers `true` for `'(,)'::int4range @> NULL`, where the strict operator must give NULL. Of 23 recent query-processing fixes in PostgreSQL master, 20 change what a query returns, but only 9 of them crash or raise an internal error. We present Arbor, a fuzzer that keeps every generated query as a typed derivation tree, so that metamorphic oracles (TLP, NoREC, expression rewrites, CTE and table fences, row-level security) can rewrite single nodes of known type at any query level, and triage reduces failures on the same trees. Plan variants and the last release serve as two more oracles. An LLM reads recent commits and proposes grammar alternatives and equivalence rewrites, which a validation gate accepts only if the server runs them, they reach the changed lines, and a rewrite never changes a result. Arbor found two previously unknown bugs, a wrong result and an assertion failure, and RQ1–RQ4 results. The main lesson is that oracle precision, not generator reach, limits a wrong-result fuzzer: a richer grammar also reaches the corners where two "equivalent" queries are not.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Information systems** → *Data management systems.*

Additional Key Words and Phrases: fuzzing, database testing, test oracles, metamorphic testing, grammar-based testing, large language models, PostgreSQL

ACM Reference Format:
Anonymous Author(s). 2026. Check the Tree: Finding Wrong Results in PostgreSQL with Typed Derivation Trees and Commit-Directed Grammar Extension. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (Format Template) (FSE (CS-5187 Term Project)).* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Suppose you run a reporting query on PostgreSQL and it returns a row that should not be there. The query does not crash and no error is raised. Nothing in the logs says that the answer is wrong. This kind of failure, a *wrong result*, is the most harmful bug a database system can have, because the user has no way to notice it.

Fuzzers for database systems mostly look for crashes. SQLsmith [Seltenreich [n. d.]] generates random queries from the system catalog, and SQUIRREL [Zhong et al. 2020] mutates SQL with coverage feedback. Both detect a bug when the server crashes or an assertion fails. To find wrong results, a fuzzer needs a *test oracle* that knows the right answer. Differential testing compares two systems [Slutz 1998]; metamorphic oracles such as Ternary Logic Partitioning (TLP) [Rigger and Su 2020b], NoREC [Rigger and Su 2020a] and equivalent expression transformation [Jiang and Su 2024] compare a query with a rewritten query that must return the same rows. These oracles rewrite the SQL text of a query, so each tool builds its own small query generator that the oracle understands.

We focus on a gap between the two lines of work. Grammar-based generators reach deep into the planner but do not know which parts of a query can be rewritten safely. Oracle-based tools

FSE (CS-5187 Term Project), 2026, IIT Guwahati
2026. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

```
SELECT c,
       '(,)'::int4range @> c       AS contains,   -- must be NULL when c is NULL
       NOT ('empty'::int4range @> c) AS not_empty  -- must be NULL when c is NULL
FROM (VALUES (NULL::int), (1)) v(c);

  c   | contains | not_empty
------+----------+----------
 NULL | t        | t          ← wrong: the planner folded the call to a constant
 1    | t        | t
```

Fig. 1. A wrong result in PostgreSQL 17, 18.6 and master that Arbor found: the planner support function for range containment returns `true` or `false` for a NULL element, where the strict operator gives NULL. No error or crash shows it; only an oracle that compares two equivalent queries does.

know how to rewrite, but their generators cover a small part of SQL. And neither knows what changed in the code yesterday: the planner of PostgreSQL master changes every week, and most new bugs live in the newest code.

Figure 1 shows a real wrong result that Arbor found. Since PostgreSQL 17, the planner replaces `range @> elem` by the constant `true` when the range is unbounded on both sides, and by `false` when it is empty. The operator is strict, so for a NULL element the correct answer is NULL. The constant is wrong under `NOT`, in `IS NULL` tests, in `CASE`, and in outer joins, where unmatched rows carry NULLs. We reproduced the bug on the release 18.6 and on master. Arbor found it with a typed rewrite of the derivation tree (a scalar subquery around the range), in the first 150 test cases the rewrite ran.

This is not a single case. Between 31 July and 16 October 2026, PostgreSQL master received 23 fixes in query processing that came with a regression test and still revert cleanly (Section 6.8). When we put each bug back, 20 of them change what a query returns: 8 return wrong rows, 5 raise internal errors, 4 fail an assertion, and 3 raise or drop an error message. Only 9 of these 20 can be seen by a crash-only fuzzer.

Closest to us is query plan guidance [Ba and Rigger 2023], which steers a generator toward new plan shapes, and Fuzz4All [Xia et al. 2024], which uses a large language model (LLM) as the generator. However, plan guidance uses the oracles of SQLancer and their small query language, and an LLM that writes every test case is slow and produces many invalid queries. Neither connects the generator to the code that changed.

We present Arbor, a grammar-aware fuzzer for PostgreSQL built on three ideas:

(1) A *typed grammar* generates SQL as derivation trees. Every node knows its type and its place in the query (Section 4.1).
(2) *Tree-based oracles* rewrite the derivation tree, not the text: TLP on `WHERE`, `HAVING` and aggregates at any query level, NoREC, typed expression and predicate rewrites, CTE materialisation, table fences, a row-level security leak check and a cardinality-estimate check. A plan-variant oracle and a cross-version oracle compare the same query under other planner settings and on the last release (Section 4.2).
(3) *Commit-directed extension*: an LLM reads a recent commit and proposes grammar alternatives, seed queries and new equivalence rewrites; a validation gate accepts them only if the server runs them, they execute the lines the commit changed, and a rewrite never changes a result (Section 4.5).

Arbor then reduces every finding on its derivation tree and its tokens, localizes wrong results to a planner path or to an always-applied rewrite, and drafts the upstream report (Section 4.4).

First results. In a four-minute smoke run on a server with the 23 bugs put back, Arbor rediscovered an assertion failure in `JSON_TABLE` that we had found earlier on master, and its triage exposed two false-positive classes in our own oracles, which we fixed (Section 6.7). RQ1–RQ4 numbers follow from the full campaigns. The lesson so far is that oracle precision, not generator power, limits a wrong-result fuzzer: a generator that reaches more of SQL also reaches more corner cases where two "equivalent" queries are not equivalent.

Contributions.

- A typed SQL grammar for PostgreSQL whose derivation trees serve generation, mutation, metamorphic rewriting and reduction (Section 4).
- Metamorphic oracles on derivation trees, including a row-level security leak check, plus plan-variant and cross-version oracles, with the precision of each (Section 6.7).
- A commit-directed LLM extension whose validation gate checks validity, reachability and diff coverage on the server, and tests LLM-proposed equivalence rewrites on values and queries before the oracle uses them (Section 4.5).
- A benchmark of 23 real PostgreSQL bugs put back into one server, with a single-bug server per bug for attribution (Section 6.8).
- Two previously unknown PostgreSQL bugs, one wrong result and one assertion failure, with patches (upstream status; Section 6.7), and the tool and all data (Section 10).

2 BACKGROUND

Derivation tree. A grammar derives a query by choosing one alternative per rule. The *derivation tree* $T(Q)$ of a query Q records these choices: each inner node is a rule with the alternative it took, and the leaves are SQL tokens. Reading the leaves from left to right gives the text of Q.

Wrong result. A query has a wrong result when the server returns a multiset of rows $[\![Q]\!]$ that differs from what the SQL standard and the PostgreSQL documentation define, without an error. Typical causes are planner rules that are correct only under an assumption that does not hold, for example that a column cannot be NULL.

Metamorphic oracle. A metamorphic oracle builds a second query Q' with a known relation to Q, usually $[\![Q]\!] = [\![Q']\!]$, and reports a bug when the relation fails. TLP [Rigger and Su 2020b] uses $[\![Q]\!] = [\![Q[p]]\!] \uplus [\![Q[\neg p]]\!] \uplus [\![Q[p \texttt{ IS NULL}]]\!]$ for a predicate p added to `WHERE`. NoREC [Rigger and Su 2020a] compares the row count of `SELECT * FROM t WHERE p` with the number of rows on which p is true when p is evaluated in the select list, where the planner cannot use it to choose a plan.

Plan variant. PostgreSQL lets a session switch planner methods off, for example `SET enable_hashjoin = off`. A *plan variant* is the same query under such a setting. All plan variants of a deterministic query must return the same multiset. This is differential testing inside one server.

Assertion build. PostgreSQL built with `-enable-cassert` checks internal invariants; a failed `Assert` aborts the backend and logs a `TRAP` line. Release builds skip these checks, so such a failure often means silent corruption or a wrong result in production.

3 PROBLEM STATEMENT

We now state the problem precisely. Let G be a typed grammar for the SQL dialect of PostgreSQL and S a database schema. A *test case* is a sequence of statements derived from G over S, run in one transaction that is rolled back at the end. An *oracle* maps the server's response to a test case to *pass* or *bug candidate*.

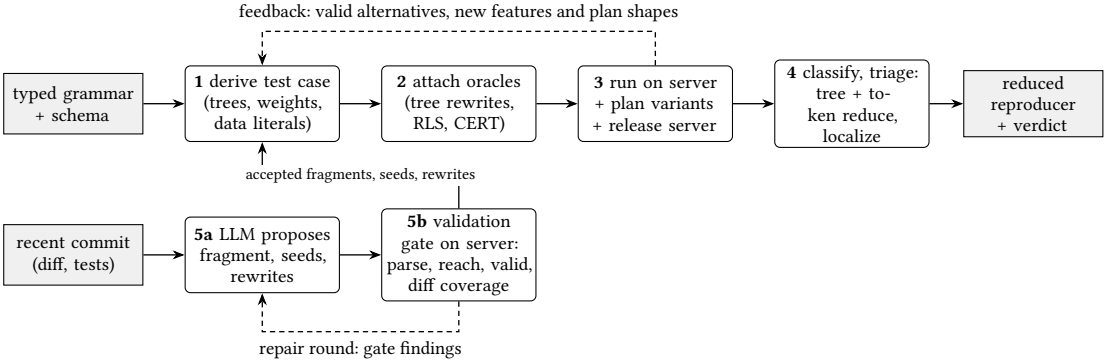


Fig. 2. Overview of Arbor. The top row is the fuzzing loop: test cases are derived from the typed grammar, oracles are attached as rewrites of the derivation tree, and the server runs each query under several settings. The bottom row runs before a campaign: an LLM turns a recent commit into grammar alternatives, and the validation gate accepts them only when the server runs them and they reach the changed lines.

Sound rewrites. A tree rewrite r maps $T(Q)$ to a second tree. It is *sound* for Q if $[\![Q]\!] = [\![r(Q)]\!]$ holds on every correct server. A rewrite that is sound only under conditions (for example, only if no `LIMIT` without `ORDER BY` appears below the rewritten node) must check them on the tree before it is applied. An unsound rewrite produces *false positives*: candidates that are not bugs.

Goal. Given a fixed time budget B and one server, find as many distinct bugs as possible, where a bug is a crash, a failed assertion, an internal error (SQLSTATE class `XX`), or a wrong result, while keeping the *precision* of each oracle high:

$$\mathit{precision}(o) = \frac{|\{c \in C_o : c \text{ is a real bug}\}|}{|C_o|} \tag{1}$$

where C_o is the set of distinct candidates oracle o reported, after de-duplication by signature (Section 4.4).

Why not let an LLM write the test cases? An LLM can write SQL that targets a feature, but at a few seconds per call it produces about a hundred test cases in four minutes, where the grammar arm of our smoke run produced 5,516 (Section 6.8). LLM-written queries are also not derivation trees, so the tree-based oracles cannot rewrite them. We use the LLM to extend the grammar instead: a few calls per commit, after which the grammar generates thousands of variations.

4 APPROACH

Figure 2 shows the steps of Arbor. The main idea is to keep every generated query as a typed derivation tree until it has been run, so that oracles can rewrite single nodes of known type and position instead of SQL text. The tree has four jobs: generation builds it, mutation re-derives one subtree, the oracles rewrite one typed node, and triage reduces it by replacing subtrees with their smallest derivation. The fuzzing loop (Steps 1–4) runs thousands of test cases per minute; its expensive part is running each query several times, once per oracle. The LLM loop (Step 5) runs a few times per commit, before the campaign, and extends both the grammar and the oracle.

4.1 Step 1: Derive Test Cases from a Typed Grammar

We wrote a grammar for the SQL that PostgreSQL 18 and master accept, in a small format of our own (`.pgg`). It has 78 rules with 421 alternatives over 13 SQL types. A rule may be parameterised by a type, as in `expr<T>`, so a derivation only combines expressions that type-check. Weights on alternatives set how often each is chosen, and *builtins* written in Python handle what a context-free grammar cannot: `@col(T)` picks a visible column of type T in the current scope, `@table` binds a fresh alias, and `@group_keys` turns the current query level into a grouped one, after which only grouped columns and aggregates are visible. The schema has 9 relations with partitions, inheritance, a view, indexes of four access methods (B-tree, GIN, GiST, BRIN) and NULLs in most columns.

A test case has up to two setup statements (DDL, DML, `ANALYZE`, `SET LOCAL` of a planner setting) followed by one or two queries. While deriving, the engine wraps the tokens of every node in invisible markers, so after rendering it knows the exact token span of each node. Half of the test cases are new derivations; the other half replay a test case from the corpus and re-derive one subtree. A test case enters the corpus when it produces a new bug signature, covers a new SQL feature, a new pair of features, or a new plan shape, the digest of the node types in `EXPLAIN` output [Ba and Rigger 2023].

Feedback-weighted alternatives. Every alternative carries a multiplicative weight. When a test case enters the corpus, the weights of all alternatives in its trees grow by $e^{\eta(1-\bar{r})}$; when a statement fails, the weights of its tree shrink by $e^{\eta(-0.5-\bar{r})}$, where $\bar{r}$ is a running average of the reward and $\eta = 0.1$. Weights stay in $[0.2, 5]$, so no alternative disappears.

Literals from the data. Random constants rarely select a real share of rows: `c0 = 4711` selects nothing and `c0 < 4711` everything. Arbor asks the server once for the minimum, maximum and the 10–90% quantiles of every column. Some literals then come from this *data profile*: the boundaries of the observed interval and of the type (min−1, max+1, 0, −0, NaN, ±Infinity, the empty array), and, through the builtin `@dlit(T)`, a quantile of the column that the predicate compares, so that `col < @dlit` selects between 10% and 90% of the rows.

We do not claim grammar-based generation as new; the typing, the scoping builtins and the node spans are what the later steps need.

4.2 Step 2: Attach Oracles as Tree Rewrites

For each query, Arbor picks up to one metamorphic check and stores it with the test case as a pair of queries that must return the same multiset. All checks are built on the derivation tree:

TLP on WHERE. Arbor picks a `SELECT` whose result is used as a multiset: the query itself, a derived table, a CTE body, a set-operation arm, or an `IN`, `EXISTS` or `LATERAL` subquery. Its `WHERE p` is partitioned into p, `NOT p` and `p IS NULL`, and the three copies are combined with `UNION ALL`. Because the rewrite edits one node, it also works inside a nested query, where text-based TLP does not apply.

TLP on HAVING. The same on the `HAVING` clause. The unpartitioned side keeps `HAVING TRUE`: without `GROUP BY`, a `HAVING` clause is what makes a level a single-group aggregate.

NoREC. `count(*)` with `WHERE p` against the sum of `CASE WHEN p THEN 1 ELSE 0 END` over the same `FROM`.

TLP on aggregates. A global aggregate whose items are all `MIN`, `MAX`, `SUM`, `COUNT`, `BIT_*` or `BOOL_*` calls equals the combination over the three partitions: the `MIN` of the `MIN`s, the `SUM` of the `COUNT`s, and so on.

Expression rewrites. One typed expression node e becomes an equivalent form: `COALESCE(e,e)`, `(SELECT e)`, `CASE WHEN TRUE THEN e END`, $e + 0$ and $e \cdot 1$ for numbers,

`e || ''` for text, and `NOT NOT p`, `p AND TRUE`, `p OR FALSE` and `CASE WHEN p THEN TRUE ELSE FALSE END` for predicates.

Predicate rewrites. `x IN (a, b)` becomes `(x = a) OR (x = b)`, which PostgreSQL 18 turns back into `= ANY` (the OR-to-ANY transformation); `a < b` becomes `NOT (a >= b)`, which three-valued logic keeps exact; and `x BETWEEN l AND h` becomes `x >= l AND x <= h`.

NOT NULL conjunct. `WHERE p` becomes `WHERE p AND c IS NOT NULL` for a NOT NULL column c that p already uses, which targets the planner's NOT NULL simplifications. The rewrite is skipped under outer joins and for generated columns, where a NOT NULL column can still be NULL.

ORDER BY removal. The statement's own `ORDER BY` is dropped; the result is compared as a multiset.

CTE materialisation. `MATERIALIZED` is toggled on one CTE.

Table fence. One table reference becomes `(SELECT * FROM t OFFSET 0)`, which stops the planner from pushing predicates into it, removing it, or pruning its partitions.

Row-level security. Five tables carry a row-level security policy. The query runs as a restricted role, under the policies, and as a superuser with every protected table t replaced by `(SELECT * FROM t WHERE policy)`. A difference means that the policies let a row through or hide one they should not. One file defines the policies for both the server and the rewrite.

Cardinality estimates. Following CERT [Ba and Rigger 2024], adding the `WHERE` clause to a query must not raise the planner's row estimate. This check compares `EXPLAIN` estimates, not results; a violation is a *performance* candidate, reported apart from bugs.

A check is attached only if its rewrite is sound for that query (Section 3): no `LIMIT` without `ORDER BY`, no volatile function, no order-dependent aggregate, and, for partitioning under `DISTINCT` or grouping, only key types whose equal values print the same text.

4.3 Step 3: Run the Test Case Several Ways

The server runs each test case in a transaction with one savepoint per statement and rolls it back at the end, so test cases cannot affect each other. For each query that is deterministic, Arbor also runs: (i) both sides of the attached check; (ii) the query under two of 18 *plan variants* (for example hash join only, no hash aggregation, forced parallel plans, partitionwise join on, no eager aggregation, JIT on), chosen at random per query; and (iii) the whole test case on a server of the last release, PostgreSQL 18.6, in a database of its own for each worker (the *cross-version oracle*). Results are compared as order-insensitive digests of the result text. The digests drop choices that no plan controls: trailing zeros of equal `numeric` values (`2.0` and `2.00`), the sign of float zero, and, for aggregates over floats, the last digits of finite values.

4.4 Step 4: Classify, De-duplicate, Triage

The oracle reads the statement results and the server log. A `TRAP` line is an assertion failure; a lost connection without one is a crash; an error of class `XX` is an internal error; a confirmed mismatch of any digest is a wrong result. A metamorphic mismatch counts only if it repeats when both sides run again. Crash evidence in the server log counts only if it carries the process id of the test case's own backend (or of a parallel worker of its query), so a crash caused by another session is never blamed on the test case that happened to be running. Each candidate gets a *signature*: the assertion location, the normalised backtrace or error message, or, for wrong results, the oracle and the shape of the query. Only the first test case per signature is saved.

Triage then reproduces the candidate three times on a fresh connection and reduces it in two stages. *Tree reduction* works on the derivation trees that every failure of the grammar arm stores: it drops a statement, drops one repetition, or replaces a subtree by the smallest derivation of the same rule and type, and replays the trees through the generator, which re-binds names, aliases and types. Every candidate is therefore valid SQL, and the server's time goes to the question whether the failure remains. *Token reduction* follows: it removes whole statements with delta debugging [Zeller and Hildebrandt 2002], then removes clauses, list items, and parenthesised groups of the failing statement level by level over its parenthesis tree [Misherghi and Su 2006], and finally single tokens. A reduction step is kept only if the same failure occurs: the same assertion, the same normalised message, or the same oracle and plan variant. For a metamorphic candidate both queries of the check are reduced together, so they stay a valid pair. We do not claim these reduction algorithms as new.

Localization. For a reduced wrong result, Arbor runs the queries again under every plan variant. If a variant removes the disagreement, the bug is in the code path that setting switches, and the report names the setting and its source files. If no variant removes it, the cause is an always-applied rewrite, constant folding, a planner support function, or expression evaluation. The output is a short reproducer, the rows each side returned, the localization, a verdict template, and a draft report in the shape that the PostgreSQL bug form asks for.

4.5 Step 5: Extend the Grammar from Recent Commits

Step 5a: propose. Arbor lists the commits of PostgreSQL master that change the optimiser, the executor, the parser or the SQL-callable functions and that add regression tests. For each one it builds a digest: the message, the changed functions, the SQL that the commit added to the tests, and the *diff-coverage targets*, the changed lines of each C file. An LLM gets the digest together with a fixed system prompt: the `.pgg` format, the builtins, the conventions of the grammar (every operator parenthesised, columns only through `@col`, no volatile functions), the schema and the whole current grammar. Because the system prompt is the same for every commit, it is cached. The model answers with structured output: a rationale, a grammar fragment of new alternatives, one to five seed test cases, the feature names it targets, and up to three *equivalence rewrites*: templates over one expression of a given type, such as `CASE WHEN {e} IS NULL THEN NULL ELSE {e} END`, that the model claims equal `{e}` for every value. The commit miner never shows the model a fix that the benchmark of Section 6.8 reverts.

Step 5b: validate. No model output reaches the campaign unless the server agrees with it. The validation gate checks, in order: (i) *parse*: the fragment parses together with the grammar, adds no types, and uses only known builtins; (ii) *reachability*: when Arbor samples 150 test cases with the new alternatives boosted, they are used at least five times; (iii) *validity*: at least 60% of the statements that use them run without error on the server; (iv) *seeds*: each seed runs without error; and (v) *diff coverage*: with a gcov build of the server, the sample executes more of the commit's changed lines than the same sample without the fragment. If a check fails, its findings (the parse error, the most frequent error messages, the lines still uncovered) go back to the model, which repairs its answer, for up to three rounds. Every round is a line in a ledger with the tokens it used, its cost and the full report of the gate.

A proposed rewrite is a claim about semantics, so the gate tests it twice. First, the server evaluates `rewrite(v) IS NOT DISTINCT FROM v` for NULL, the edge values of the type and quantiles of the data. Then Arbor applies the rewrite alone to the expressions of 150 test cases, with confirmation re-runs. It accepts the rewrite only if no value and no query shows a difference. Sampling alone is not enough: `{e} IS TRUE` passed 109 sampled checks, because most predicates sit where NULL

and false filter the same rows, and the value probe rejected it with e = `NULL`. An accepted rewrite becomes a new expression rewrite of the oracle in Step 2.

The gate is also a small fuzzing campaign aimed at the commit: a crash or wrong result found while sampling is recorded like any other candidate.

5 IMPLEMENTATION

Arbor is about 10,500 lines of Python with 138 unit tests, plus the 628-line grammar. It drives PostgreSQL through `psycopg`, reads the server log for assertion and crash evidence, and runs every trial as a resumable process whose events, timeline and failures are written as JSON. The parts are: Its parts are the `.pgg` parser and derivation engine (scopes, type variables, node spans, the `MINIMAL` marker); the generator (corpus, alternative weights, data profile); the executor (savepoints, plan variants, metamorphic, RLS and CERT checks, and a cross-version wrapper); triage (reproduction, tree and token reduction, localization, reports, bisection, the precision ledger); the benchmark scripts with `bench verify` and `bench attribute`; and the LLM arm (commit miner, prompts, a client that stores every response for replay, the validation gate and the ledger).

6 EVALUATION

We ask four questions.

RQ1 Does the typed grammar reach more of PostgreSQL than a random-mutation generator in the same time: more valid statements, more covered lines and branches, more SQL features and plan shapes?

RQ2 Which bugs does Arbor find, and how precise is each oracle?

RQ3 On real bugs put back into PostgreSQL, which configuration finds how many, and how fast?

RQ4 What does the commit-directed LLM extension add, and what does it cost in tokens, money and repair rounds?

6.1 Subject and Benchmark

All servers are built from one commit of PostgreSQL master, `26bd08fb` of 16 October 2026, with assertions on. For **RQ3** we built a benchmark of real bugs in the style of Magma [Hazimeh et al. 2020], but by reverting fixes instead of porting bugs forward. We took every commit between 31 July and 16 October 2026 that:

(1) changes the optimiser, executor, parser, rewriter or SQL-callable functions, and says in its message that it fixes a bug, because RQ3 is about bugs that reach users through SQL;
(2) adds a regression test, because its SQL is our proof that the bug can be triggered;
(3) reverts cleanly on the base commit, alone and together with the others, and the server still builds.

This gives 23 fixes. We build three kinds of server: the *fixed* server (the base), one *solo* server per bug (the base with only that fix reverted), and the *injected* server (all 23 reverted), on which the campaigns run. For each bug we wrote a self-contained proof of concept (PoC) from its regression test and ran it on the fixed and the solo server (Table 1). Twenty bugs change the result of their PoC. The other three change only the plan, or need an equality operator that returns NULL for non-NULL inputs, which no built-in type has; they leave the denominator. On the injected server every PoC shows the same symptom as on its solo server, so no injected bug hides another.

6.2 Configurations

Every configuration (*arm*) uses the same server, schema, executor, oracles, time budget and per-trial random seed; the arms differ only in the generator and in what the generator can attach.

Table 1. The re-injected bug benchmark. Symptom: what the PoC shows on the solo server compared with the fixed server. A crash-only fuzzer can see the first two rows; the others need an oracle that knows the expected result or the expected error.

Symptom	Bugs
assertion failure	4
internal error (`XX000`)	5
wrong result	8
missing error (the bug returns rows instead of an error)	2
wrong error	1
observable, total	20
plan only or not triggerable with built-in types	3

Random. A mutation-based generator over the same schema that edits seed queries at the token level. It cannot attach metamorphic checks, because it has no derivation trees.

Grammar. Step 1 alone: the typed grammar with feature-guided corpus, no metamorphic checks, no plan guidance.

Grammar+Oracles. Steps 1–4: metamorphic checks (including the RLS and CERT checks) on half the queries, plan-shape guidance, feedback-weighted alternatives and literals from the data profile. In **RQ1**, Grammar+Oracles, flat is the same arm without the weights, which isolates their effect.

+LLM. Grammar+Oracles plus the fragments, seeds and equivalence rewrites that passed the validation gate. For **RQ4** we also run +LLM fragments and +LLM seeds alone. The LLM never sees the 23 benchmark fixes: the commit miner excludes them.

All arms run with the plan-variant oracle and, in RQ3, the cross-version oracle, because these do not depend on the generator.

6.3 Measurements

RQ1. Statement validity (the share of statements that run without error), line and branch coverage of the backend from gcov, distinct SQL features from a catalogue of 148 and distinct plan shapes, over time.

RQ2. Distinct bug candidates per oracle after signature de-duplication, and the verdict of each after triage: real, duplicate, false positive, not a bug, or flaky (Equation 1); we give precision before and after fixing each false-positive class. CERT candidates are performance issues and are counted apart.

RQ3. For each benchmark bug and arm: the number of trials that found it and the median time to the first finding. A crash or assertion is attributed by its signature; a wrong result or error is attributed by replaying the failing test case on the fixed server and on every solo server: it belongs to bug i if it fails on solo server i and not on the fixed server.

RQ4. Share of commits whose proposal passes the gate, repair rounds needed, reasons for rejection, diff coverage gained, equivalence rewrites proposed and accepted, tokens and cost; and the **RQ1**–**RQ3** measures of the three LLM arms.

6.4 Setup

Each arm runs 5 trials of one hour on one worker each, four workers in parallel, on machine. Following Klees et al. [Klees et al. 2018], and Arcuri and Briand [Arcuri and Briand 2011], we report medians with interquartile ranges and 95% bootstrap confidence intervals, compare arms with the

Table 2. Component checks on master (single runs). Each row compares the component with the same run without it, or counts false positives of an oracle over live checks.

Component	Result
New rewrites (TLP on aggregates, predicate, NOT NULL, ORDER BY)	798 live checks of the new kinds (1,066 in all) in 1,600 test cases, 0 false positives
RLS and CERT checks	532 RLS and 58 CERT checks, 0 false positives
Data profile	column-versus-constant predicates that select 5–95% of their table: 29.3% → 58.6%
Feedback-weighted alternatives	distinct plan shapes in 1,000 test cases: 419 → 533, validity unchanged (99.7%)
Tree reduction (BUG-01 case)	final reproducer 116 → 79 tokens (406 vs. 555 server runs)
Localization (BUG-02)	"not toggled by planner settings": the planner support function, the true cause
Rewrite gate	accepted two identities (90/90, 88/88 checks equal); rejected `{e}+1` (12 mismatches) and `{e} IS TRUE` (value probe, e = NULL)
Tree replay	599 of 600 test cases replay to identical SQL in a fresh process

two-sided Mann–Whitney U test, and give the Vargha–Delaney $\hat{A}_{12}$ effect size [Vargha and Delaney 2000]. Each coverage worker writes its gcov counters to its own directory, so parallel trials do not mix coverage. The LLM is Claude Opus 5.5 with structured output and a cached system prompt of about 9,000 tokens.

6.5 Component Checks

Before the campaigns, we checked each component on its own against master (Table 2). These are single runs on one seed, so they show that a component works and is sound, not how large its effect is in a campaign; the campaigns measure that.

Observation 1. Testing a claimed equivalence on sampled queries is not enough: the false identity `{e} IS TRUE` passed 109 sampled checks and failed on the first probe value, NULL.

6.6 RQ1: Reach

Table: median (IQR) validity, lines, branches, features, plan shapes per arm, with $\hat{A}_{12}$ against Random; figure: coverage over time.

After 270 seconds of a smoke run on the injected server (one trial per arm), each grammar arm had produced 101 distinct SQL features and more than 2,000 distinct plan shapes, the random arm 17 features and 17 plan shapes. Of the 19,846 statements the Grammar arm sent, 19,780 (99.7%) ran without error. Replace with the full campaign.

6.7 RQ2: Bugs and Oracle Precision

Table: candidates and verdicts per oracle (from the triage ledger).

So far Arbor found two bugs that were not known. The first is the wrong result of Figure 1, found by the `(SELECT e)` expression rewrite. The second is an assertion failure: `JSON_TABLE` does not assign collations to its context item and `PASSING` arguments, and a row comparison over such a value fails an assertion in the executor (PostgreSQL 17, 18 and master). The four-minute smoke run of RQ3 found the second bug again without help.

Triage of the smoke run also found two false-positive classes in our own oracles, which we fixed before the campaign. TLP on `HAVING` dropped the clause on the unpartitioned side, which turns a single-group aggregate into a plain query; and `DISTINCT` over `numeric` may keep `21.000000000000000` or `21.0000000000000000000` depending on the plan, which are equal values. The smoke runs also exposed a harness error: when another session crashed the server, the crash was blamed on the test case that happened to be running. Arbor now reads the backend process id from every `TRAP` and signal line of the log and ignores evidence from other backends. Over all 18 candidates triaged so far, 3 are real and 13 are false positives or intended changes, all of them from 6 classes that we fixed; final precision per oracle, before and after fixes.

6.8 RQ3: Re-injected Bugs

Table from `pgfuzz bench attribute`: trials and median time per bug and arm.

In a 15-minute smoke run (one trial per arm), Grammar+Oracles and +LLM each found the internal error of fix `dfcf6fa7` (dummy `Append` paths for set operations) after about 160 seconds; Random and Grammar found no benchmark bug. The error message of this bug is shared by fix `8b29b8da`; replay on the two solo servers assigned it to the right one. Full result.

6.9 RQ4: The LLM Extension

Ledger summary: commits, acceptance after round 1/2/3, rejections by gate check, diff coverage gain, tokens, cost; and the ablation of the three LLM arms on **RQ1**–**RQ3**.

7 DISCUSSION

Is a larger grammar always better? No. A larger grammar reaches more corner cases of the SQL semantics, and some of these break the assumptions of an oracle. Both false-positive classes of our smoke run came from SQL that a small generator would not produce: `HAVING` without `GROUP BY`, and `DISTINCT` over a computed `numeric` with values of different scale. The cost of a larger grammar is paid in oracle precision, and every rewrite needs a soundness check on the tree.

Why keep the tree after generation? Because it does four jobs for the price of one. Mutation re-derives a subtree, the oracles rewrite one typed node, and reduction replaces a subtree by its smallest derivation; in each case the generator replays the tree and re-binds names and types, so the result stays valid SQL. On the BUG-01 case, tree reduction followed by token reduction gave a 79-token reproducer where token reduction alone gave 116.

Why rewrite trees instead of text? A text rewrite must parse the query again to find the clause it changes, which is hard for SQL with nested subqueries, CTEs and set operations. On the tree, the clause is one node with a known span. This is what lets Arbor apply TLP inside a derived table or a `LATERAL` subquery.

Why compare with the last release? Plan variants cannot catch a wrong result that every plan shares, for example a constant folded by the planner before any plan is chosen, as in Figure 1. The release does not have the newest planner rules, so it disagrees with exactly the bugs that new code brings. It also disagrees with intended changes in behaviour, which triage must recognise.

Can an LLM be trusted to write oracle rewrites? Not on its word. A rewrite is a claim about semantics, and a wrong claim produces false positives in every campaign that uses it. The validation gate therefore tests each one on concrete values and on live queries, and Observation 1 shows why both are needed.

Does the LLM need to be good at SQL?. It needs to know PostgreSQL features well enough to name the SQL that reaches a commit, which is what its training data contains. The validation gate does not trust it: an alternative that does not parse, is never derived, fails on the server, or misses the changed lines is rejected and explained back to the model. How often the repair succeeds is part of **RQ4**.

8 THREATS TO VALIDITY

Internal threats. Fuzzing is random, and one run proves little. We guard against this as follows:

- every arm runs 5 trials with the same per-trial seeds, and we report medians, interquartile ranges, Mann–Whitney U tests and $\hat{A}_{12}$ effect sizes [Klees et al. 2018; Vargha and Delaney 2000];
- arms share the server build, schema, executor, oracles and budget, so a difference comes from the generator;
- each trial runs in its own server and, for the release comparison, in its own database, so parallel trials cannot see each other's tables;
- a metamorphic mismatch counts only if it repeats, and every candidate is reproduced three times on a fresh connection before triage.

Our own oracles can be wrong. Two false-positive classes reached the smoke run (Section 6.7); we fixed them and added regression tests, but others may remain. We therefore report the verdict of every candidate, not only the real bugs. Verdicts are made by us and may contain mistakes; we release the reduced reproducers so that others can check them. An accepted LLM rewrite is tested, not proven: the value probe and 150 test cases can miss an input on which it is not an identity. Such a rewrite would show up as a false-positive class in triage, which we report like any other. Attribution in RQ3 replays a test case on single-bug servers; a finding that needs two injected bugs at once is reported as unattributed, not credited to an arm.

External threats. We test one system, PostgreSQL, at one commit, with one schema of 9 relations and one set of row-level security policies that we chose. Bugs that need other data shapes, extensions or concurrent sessions are out of reach. The 23 benchmark bugs are recent fixes in query processing with regression tests; bugs that were never fixed, or whose fix does not revert cleanly, are missing, and the benchmark may favour the features that recent PostgreSQL development touched. The LLM results hold for one model at one point in time; we store every prompt and response, so the ledger can be replayed without the model, but a new run may differ.

Construct threats. Line and branch coverage measure what code ran, not whether an oracle could have seen a bug there. A bug "found" in RQ3 means a test case failed on its solo server and passed on the fixed one; the failure may differ from the PoC's symptom, for example a wrong result where the PoC shows an internal error, and we still count it, because the same fix removes it. For the LLM, a fragment that passes the validation gate is valid and reaches the changed lines; it is not proof that it tests the changed behaviour.

9 RELATED WORK

Metamorphic oracles for database systems. Closest to us is the work of Rigger and Su: pivoted query synthesis [Rigger and Su 2020c], NoREC [Rigger and Su 2020a] and TLP [Rigger and Su 2020b], all in SQLancer, and equivalent expression transformation by Jiang and Su [Jiang and Su 2024]. These tools found hundreds of bugs in widely used systems. They build the follow-up query from strings that their own small generators produced. We share their oracles, but apply them to

typed derivation trees of a grammar that covers much more of PostgreSQL, at any query level, and we report the precision of each oracle on that larger language.

Guidance for database fuzzing. Query plan guidance [Ba and Rigger 2023] steers SQLancer toward queries with new plan shapes. SQUIRREL [Zhong et al. 2020] mutates an intermediate representation of SQL with coverage feedback to find crashes, and SQLRight [Liang et al. 2022] combines such coverage-guided mutation with the logic-bug oracles of SQLancer. CERT [Ba and Rigger 2024] finds performance issues by checking that a more restrictive query never gets a larger cardinality estimate; we include its check as one oracle. Directed greybox fuzzing [Böhme et al. 2017] steers a fuzzer toward chosen code locations. We use plan shapes as one of four reasons to keep a test case and learn alternative weights from feedback, and we direct the campaign toward the commits that changed the code under test, at the level of SQL features rather than branches.

Grammar-based and differential SQL testing. RAGS [Slutz 1998] generated random SQL and compared the results of several database systems. SQLsmith [Seltenreich [n. d.]] generates random queries from the catalog of a running PostgreSQL and has found many crashes. We compare one server with itself under 18 plan variants and with its last release, which avoids most dialect differences, and we need a typed grammar instead of the catalog because the oracles must know the type and scope of every node.

LLMs for fuzzing. Fuzz4All [Xia et al. 2024] uses an LLM as the input generator for compilers and language tools, and updates its prompt during fuzzing. We use the LLM differently: it writes grammar alternatives for one commit, a server-side validation gate rejects what does not run or does not reach the changed lines, and the grammar then produces the test cases. The LLM is called a few times per commit, not once per test case, and it can also extend the oracle with equivalence rewrites, which the gate tests on values and on live queries before use.

Benchmarks and evaluation of fuzzers. Magma [Hazimeh et al. 2020] ports real bugs into fixed versions of real programs and measures which fuzzers trigger them; Klees et al. [Klees et al. 2018] showed how fuzzing evaluations go wrong without repeated trials and statistics. We follow both: our benchmark reverts real fixes in one server, attributes each finding to its bug by replay on single-bug servers, and every arm runs repeated trials.

Test-case reduction. Delta debugging [Zeller and Hildebrandt 2002] and hierarchical delta debugging [Misherghi and Su 2006] reduce failing inputs. We use both and add paired reduction for metamorphic checks, where the two queries of a check must shrink together to stay equivalent.

10 CONCLUSION

Wrong results are the bugs of a database system that users cannot see. Of 23 recent fixes in PostgreSQL query processing, 20 change what a query returns and only 9 of them crash or raise an internal error. We presented Arbor, which generates SQL as typed derivation trees and rewrites those trees to build metamorphic checks. It compares each query with its plan variants and with the last release, and it extends its grammar from recent commits with an LLM whose proposals a validation gate checks on the server. Arbor found two previously unknown bugs in PostgreSQL and RQ1–RQ4 summary, same numbers as the abstract. The precision of the oracles, not the reach of the generator, decided how useful its findings were.

DATA AVAILABILITY

The replication package is available at anonymous mirror URL. It contains the source of Arbor with its unit tests, the grammar and the schema. It also contains the benchmark: the 23 fix commits, their

PoCs and the scripts that build the fixed, solo and injected servers. Finally, it holds the experiment configurations, the run logs and reports of every trial, the triage verdicts, and the LLM ledger with every prompt and response, so that **RQ4** can be replayed without calling the model.

USE OF GENERATIVE AI

[TODO: Confirm and finalise with all authors.] Arbor calls Claude Opus 5.5 (Anthropic) as part of its method (Section 4.5); every prompt and response is in the replication package. A generative AI assistant (Claude Code with Claude Opus 5.5) was also used to write parts of the source code and to draft parts of this paper. The authors checked every statement, every citation and every number. All numbers reported here were produced by running the tool and scripts.

REFERENCES

Andrea Arcuri and Lionel Briand. 2011. A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*. 1–10. https://doi.org/10.1145/1985793.1985795

Jinsheng Ba and Manuel Rigger. 2023. Testing Database Engines via Query Plan Guidance. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2060–2071. DOI: verify.

Jinsheng Ba and Manuel Rigger. 2024. CERT: Finding Performance Issues in Database Systems Through the Lens of Cardinality Estimation. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 133:1–133:13.

Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed Greybox Fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. https://doi.org/10.1145/3133956.3134020

Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. 2020. Magma: A Ground-Truth Fuzzing Benchmark. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4, 3, Article 49 (2020). https://doi.org/10.1145/3428334

Zu-Ming Jiang and Zhendong Su. 2024. Detecting Logic Bugs in Database Engines via Equivalent Expression Transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 821–835.

George T. Klees, Andrew Ruef, Benjamin Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. https://doi.org/10.1145/3243734.3243804

Yu Liang, Song Liu, and Hong Hu. 2022. Detecting Logical Bugs of DBMS with Coverage-based Guidance. In *31st USENIX Security Symposium (USENIX Security)*. 4309–4326.

Ghassan Misherghi and Zhendong Su. 2006. HDD: Hierarchical Delta Debugging. In *Proceedings of the 28th International Conference on Software Engineering (ICSE)*. 142–151.

Manuel Rigger and Zhendong Su. 2020a. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. https://doi.org/10.1145/3368089.3409710

Manuel Rigger and Zhendong Su. 2020b. Finding Bugs in Database Systems via Query Partitioning. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020). https://doi.org/10.1145/3428279

Manuel Rigger and Zhendong Su. 2020c. Testing Database Engines via Pivoted Query Synthesis. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 667–682.

Andreas Seltenreich. [n. d.]. SQLsmith: A Random SQL Query Generator. https://github.com/anse1/sqlsmith. Accessed: 2026-10-23.

Donald R. Slutz. 1998. Massive Stochastic Testing of SQL. In *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB)*. 618–622.

András Vargha and Harold D. Delaney. 2000. A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* 25, 2 (2000), 101–132. https://doi.org/10.3102/10769986025002101

Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. https://doi.org/10.1145/3597503.3639121

Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE Transactions on Software Engineering* 28, 2 (2002), 183–200.

Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. 2020. SQUIRREL: Testing Database Management Systems with Language Validity and Coverage Feedback. In *Proceedings of the 2020 ACM SIGSAC Conference*

on Computer and Communications Security (CCS). 955–970.